



JAVA PROGRAMMING (340)

STATE – 2020

Production Portion:

Program 1: Golf Course Management _____ (350points)

TOTAL POINTS _____ (***350 points***)

**Judge/Graders: Please double check and verify all
scores and answer keys!**

Property of Business Professionals of America.
May be reproduced only for use in the Business Professionals of America
Workplace Skills Assessment Program competition.

You will have 90 minutes to complete your work.

Your name and/or school name should *not* appear on any work you submit for grading. Write your contestant number in the provided space. Staple all pages in order before you turn in your test.

To submit your project:

1. Create a folder on the flash drive using your contestant number as the name of the folder.
2. Copy your entire solution/project into this folder, including source code.
3. Ensure all the files required to run your program are present on the flash drive and will run on the flash drive.

Use the default package for the Java projects you create.

Golf Course Management

You have been hired by a local golf course to write a program to manage the golfer's data. All the data will be stored on a text file (provided) named "Players.txt". The file will contain a comma delimited line for each player that will provide their name and a series of scores. The programmer can assume that each record has a full name and at least one score. The file will be written in the format shown in Figure 1.

fName lName,##,##,##,##,##,##...

Figure 1

You will need to create a class named "player" based on the UML diagram provided below (figure 2). Note that the class will only have one overloaded constructor and no default constructor. The constructor will take in the comma delimited string read from the input file and parse out the name and fill the array/array list with the scores. The administrators of the course have decided to calculate the player's handicaps based 90% of the average of the player's top five rounds. If the player has fewer than 5 rounds, their handicap will base on 90% of the average of all of the rounds played.

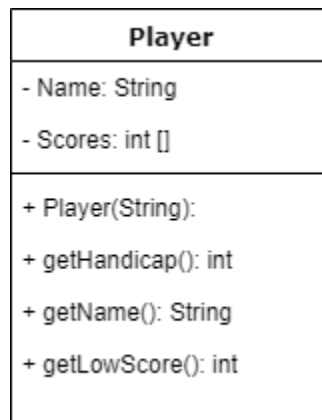


Figure 2

The main class should be name "PlayerDriver".On startup, the program should read the input file and populate a data structure containing all the player objects. The main program will need to use a menu feature that looks similar to the one below (Figure 3):

- 1- Find player handicap
- 2- List all players best round
- 3- List all players handicaps
- 0- Exit

Figure 3

All user input must be user validated. Option 1 will prompt the user to enter a name of a player. The program will search the data structure for that player. If the player is found, the program will list Name and Handicap. If the name is not found, the program should display a message notifying the user that the player was not found. Options 2 will list all the player's names and their best (lowest) scores. Option 3 will list all the players' names and their calculated handicaps. After options 1-3 are selected, the program should ask the user to press the enter key to continue.

Your program will be graded on the following criteria:

Solution and Project

The project is present on the flash drive.	_____	10 points
The project's main class is called Decoder.	_____	10 points

Program Execution

The program runs from the USB flash drive.	_____	10 points
--	-------	-----------

If the program does not execute, then the remaining items in this section receive a score of zero.

Menu is implemented.	_____	20 points
The menu and its input is user validated	_____	10 points
The program prompts for a player's name when option 1 is selected	_____	20 points
The program prints the players name and handicap for option 1	_____	20 points
The program lists all the players' lowest scores when option 2 is selected	_____	30 points
The program lists all the players' handicaps when option 3 is selected	_____	30 points
The program exits when option 0 is selected	_____	10 points

Source Code Review

The contestant number is in a comment above the start of the main class.	_____	10 points
The name of the main class is PlayerDriver.	_____	10 points

The program has a class named Player _____ 20 points

The Player class has a single overloaded constructor that takes in comma delimited string and parses it correctly _____ 30 points

The Player class has a method getHandicap that calculates the handicap _____ 30 points

The Player class has methods getName and getLowScore that work correctly (15 points each) _____ 30 points

The program uses a try-catch to handle any input file errors _____ 30 points

Code is commented throughout the program _____ 20 points

Total Points = _____ / 350 points

Suggested Solution

PlayerDriver

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.*;
import java.io.*;
/**
 * Write a description of class PlayerDriver here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class PlayerDriver
{
    public static void main(String[] args){
        String inString;
        int tmpInt, menuItem;

        ArrayList<Player> golfers = new ArrayList<Player>();

        try{

            Scanner read = new Scanner(new File("Players.txt"));
            do{
                inString = read.nextLine();
                golfers.add(new Player(inString));
            }while(read.hasNext());

            read.close();

        }catch(FileNotFoundException fnf){
            System.out.println("File was not found.");
        }

        Scanner scanner = new Scanner(System.in);
        do{
            System.out.println("Please choose from the following options");
            System.out.println("1 - Find player handicap");
            System.out.println("2 - List all players best rounds");
            System.out.println("3 - List all players handicaps");
            System.out.println("0 - Exit");
            menuItem = scanner.nextInt();

            switch(menuItem) {
```

```

case 1:
//Find Player Handicap
int index=0;
boolean found = false;
String name, tmpName;
name = scanner.nextLine();
System.out.println("Please enter the player's name: ");
name = scanner.nextLine();
do
{

    if(name.equals(golfers.get(index).getName()))
    {
        found = true;
        System.out.println(golfers.get(index).getName());
        System.out.println("Handicap: " + golfers.get(index).getHandicap());

    }
    index++;

}
while(index < golfers.size() && !found);
if(!found)
{
    System.out.println("Player, " + name + ", not found!");
}
System.out.println("Press enter to continue.");
try{
    System.in.read();
}catch(Exception e){
    e.printStackTrace();
}

break;

case 2:
//List all players best rounds
for (int i = 0; i < golfers.size(); i++){
    System.out.println(golfers.get(i).getName());
    System.out.println("Low Score: " + golfers.get(i).getLowScore());
}
System.out.println("Press enter to continue.");
try{
    System.in.read();
}catch(Exception e){
    e.printStackTrace();
}

```

```

        }
        break;
        case 3:
        //List all players handicaps
        for (int i = 0; i < golfers.size(); i++){
            System.out.println(golfers.get(i).getName());
            System.out.println("Handicap: " + golfers.get(i).getHandicap());
        }
        System.out.println("Press enter to continue.");
        try{
            System.in.read();
        }catch(Exception e){
            e.printStackTrace();
        }
        break;
        case 0:
        //Exit
        break;
    }

    }while(menuItem != 0);
}
}

```

Player

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.*;
/**
 * Write a description of class Player here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Player
{
    private String name;
    private ArrayList<Integer> scores;

    public Player(String inString){
        name = inString.substring(0,inString.indexOf(','));

        Integer tmpInt;
        inString = inString.substring(inString.indexOf(',')+1);
        String[] values = inString.split(",");
        String tmpString;
    }
}

```

```

        Integer[] intArray = new Integer[values.length];
        //ArrayList<String> tmpList = new ArrayList(Arrays.asList(values));
        for (int i = 0; i < values.length; i++) {

            tmpString = values[i];
            tmpInt = Integer.valueOf(values[i]);
            intArray[i] = Integer.parseInt(tmpString);

        }
        scores = new ArrayList<Integer>(Arrays.asList(intArray));
        Collections.sort(scores);

    }
    public String getName(){
        return name;
    }

    public void PrintScores(){
        for (Integer i:scores){
            System.out.println(i);
        }

    }

    public Integer getLowScore()
    {

        return scores.get(0);
    }

    public int getHandicap(){
        int tmpTotal=0;
        double tmpHandi;
        if(scores.size() < 5){
            for (Integer i:scores){
                tmpTotal += i;
            }
            return (int)(tmpTotal*.90)/scores.size();

        }
        tmpHandi = ((scores.get(0) + scores.get(1) + scores.get(2) +
        scores.get(3) + scores.get(4))*9)/5;
        return (int)tmpHandi;
    }
}

```